Verified Projection-Event Calculus in Lean 4: Bundled Arbitrary-Partition Pinching, Lüders Retractions, and CHSH Interoperability

Bernhard Mueller*
Pragma Research Inc., Washington, United States

July 19, 2026

Paper release: `r1556` **Released:** July 19, 2026

Abstract

Reusable formalization of finite-dimensional quantum measurement requires interfaces connecting matrix algebra, positive states, partial state updates, block decompositions, and downstream inequalities. We present a Lean 4 development for projection events over finite complex matrix algebras. A projective partition induces a bundled star-subalgebra of matrices commuting with every partition projector and a bundled complex-linear pinching map. We prove that the map is unital, positive, trace preserving, idempotent, and satisfies the commutant bimodule law; its range and fixed-point set are exactly the bundled commutant. We also verify its Hilbert–Schmidt Pythagorean and contraction identities and a trace-duality uniqueness theorem. Lüders conditioning is exposed both as a zero-totalized matrix formula and as a typed state update requiring nonzero outcome weight. Its fixed states are precisely the states certain of the event, without an extra nonzero guard, and the typed update is natural under pinching by commutant events. An event-level client maps four projections to dichotomic observables and applies a complementary operator-norm Tsirelson theorem through Mathlib's CHSH interface. We compare the resulting API with Mathlib, Physlib, recent Lean quantum-information libraries, and Isabelle/HOL. The artifact pins Lean and Mathlib revisions and prints declaration-level axiom dependencies. It does not claim a completely positive channel bundle, a Physlib correspondence theorem, rank-one classicality, or a state-expectation CHSH bound.

Keywords: formal verification, Lean 4, projective measurement, partition pinching, Lüders update, CHSH

Mathematics Subject Classification (2020): Primary 68V20; Secondary 81P15, 81P40, 46L53.

*Corresponding author: bernhard@floatingpragma.ai

1 Introduction

Formalized mathematics is most reusable when checked theorems are exposed through interfaces that downstream developments can compose. Finite quantum measurement is a useful test case. The underlying objects are finite matrices, yet a faithful development crosses projection algebra, positive and normalized states, trace pairings, partial state updates, block decompositions, linear maps, subalgebras, and operator norms. Paper proofs often move between these layers silently; a proof assistant forces every conversion and side condition to be explicit.

This paper develops a Lean 4 calculus for finite projection events. Its organizing object is an arbitrary finite projective partition of the identity. The associated pinching removes off-block components and retracts the ambient matrix algebra onto the partition commutant. Selective Lüders updates and CHSH observables are clients of the same projection interface rather than independent case studies. This design makes it possible to state precisely which results are identities of the underlying star algebra and which consume the trace, positivity, or state-normalization layers.

Lean declaration identifiers are printed throughout in monospaced type, for example `partitionPinching_range`; the underscore is part of the identifier. To locate a declaration, search for the exact printed identifier, after removing only TeX's protective backslash before each underscore, in the `EventAlgebra/*.lean` files. In the public repository these files are under `Lean/ObserverPatchHolography/Source/EventAlgebra/`; in the standalone artifact they are under `event-algebra-lean/EventAlgebra/`. Table 1 maps each module name to its subject matter, and the top-level `EventAlgebra.lean` file imports all five modules.

The surrounding formal ecosystem is already substantial. Mathlib contains an order-form CHSH/Tsirelson theorem and the reusable `IsCHSHTuple` interface [4]. Physlib's `QuantumInfo.Finite.Pinching` constructs the spectral pinching of a state as a completely positive trace-preserving map and proves commutation, fixed-point, Pythagorean, and pinching-bound results [6]. Lean-Quantum provides broad basis-independent infrastructure for states, channels, tensor products, Choi and Kraus representations, and trace inequalities [3]; Lean-QIT develops a compositional infrastructure for quantum-information theory [8]. Zhao and Yu formalize the substantially stronger CHSH rigidity theorem in Lean 4 [7]. Isabelle/HOL developments cover projective measurement and the CHSH upper bound [1, 2]. We therefore make no priority claim for projective measurement, pinching, or Tsirelson's inequality.

The checked contribution is an API-level combination with a specific delta:

(i) an arbitrary supplied projective partition, independent of a selected state or spectral resolution, together with its commutant bundled as a `StarSubalgebra`;

(ii) partition pinching bundled as a complex `LinearMap`, with exact range and fixed-point theorems, positivity, unitality, trace preservation, the commutant bimodule law, Hilbert–Schmidt geometry, and map-level uniqueness;

(iii) a typed nonzero-weight Lüders state update, an unguarded fixed-point characterization for states, and naturality of that typed update under partition pinching for commutant events;

(iv) an event-to-observable CHSH client that discharges Mathlib-compatible selfadjointness, involutivity, and cross-commutation hypotheses before applying a complementary norm-form bound; and

(v) a reproducible, version-pinned artifact with per-declaration axiom output and no proof placeholders.

The boundary of the contribution is equally explicit. The linear pinching map is not bundled here as a completely positive trace-preserving channel. No formal theorem identifies an arbitrary partition with Physlib's spectral partition, no rank-one classical-representation theorem is present, and the CHSH client proves an operator-norm upper bound rather than a state-expectation bound or a sharpness witness. These are concrete extension targets, not implicit consequences of the current source.

Section 2 fixes the mathematical and Lean interfaces. Sections 3 and 4 treat projection events and selective update. Section 5 gives the bundled partition construction and its geometry. Sections 6 and 7 describe the state functional and CHSH client. Section 8 reports the audit methodology, and Section 9 positions the development feature by feature.

2 Mathematical and formal setting

Let $\mathrm{M}_n(\mathbb{C})$ be the complex $n \times n$ matrices, with conjugate transpose $X \mapsto X^*$, trace Tr, and identity $\mathbf{1}$. A projection event is a Hermitian idempotent $P = P^* = P^2$. A state is a positive-semidefinite matrix ρ with $\mathrm{Tr}(\rho) = 1$, and the trace pairing is $\mu_\rho(M) = \mathrm{Tr}(\rho M)$.

In Lean these interfaces are predicates over Mathlib matrices: `IsEvent P` is the conjunction $P^* = P$ and $P^2 = P$; `IsState rho` is positive semidefiniteness and unit trace. For state-facing functions the development additionally defines the subtypes `ProjectionEvent n` and `StateMatrix n`. The split is intentional: algebraic lemmas remain convenient on raw matrices, while an API that claims to return a state carries that invariant in its codomain.

The project is divided into five modules (Table 1). We avoid declaration counts as a proxy for contribution; the relevant unit of comparison is a compiled interface and its downstream use.

Module	Principal interface
`Basic`	projection events, states, trace pairing, closure and Born weight bounds
`Lueders`	totalized matrix formula, typed state update, repeatability, composition, and fixed points
`PartitionPinching`	projective partitions, bundled commutant, bundled linear pinching, exact range, geometry, bimodule law, and naturality
`StateExpectation`	trace pairing bundled as a complex-linear functional, normalization and positivity
`Tsirelson`	CHSH ring identity, norm theorem, Mathlib interoperability, and event-level matrix client

Table 1: Module structure of the checked development.

Three Mathlib scopes matter operationally. `ComplexOrder` equips $\mathbb{C}$ with the partial order in which $0 \le z$ means that z is real and nonnegative. `MatrixOrder` supplies the Loewner order. `Matrix.Norms.L2Operator` activates the finite-matrix operator norm and C*-ring instances used by

the matrix CHSH theorem. The source documents these scope requirements because an apparently missing theorem can otherwise be an instance-selection problem rather than a mathematical gap.

Documentation tags classify results as *algebra-only* or *trace-dependent*. The tag is explanatory rather than a separate import hierarchy. It records whether a proof uses only ring/star/norm structure or passes through $\mathrm{Tr}(\rho M)$ and, where stated, positivity or normalization.

3 Projection events and the trace pairing

The event layer verifies the standard closure inventory. Zero, identity, and $\mathbf{1} - P$ are events; orthogonality of events is symmetric; orthogonal sums and products of commuting events are events; subevent absorption is two-sided; and an event is positive semidefinite. The declaration `IsEvent.one_sub_two_smul_involution` proves that $\mathbf{1}-2P$ is a selfadjoint involution, the adapter used in Section 7.

The trace pairing is additive, homogeneous, compatible with finite sums, and obeys the sandwich identity

$$\mathrm{Tr}(P\rho P) = \mathrm{Tr}(\rho P)$$

for idempotent P. Hermiticity of ρ and P implies that the pairing is real. Positivity of ρ and the event property imply $0 \le \mu_\rho(P)$ in Mathlib's order on $\mathbb{C}$; for a state the upper bound is $\mu_\rho(P) \le 1$.

The former declaration `bornWeight_add_of_orthogonal` had orthogonality hypotheses that its proof did not use: additivity holds for all matrices. It has been replaced by `isEvent_add_and_bornWeight_add_of_orthogonal`, whose conclusion combines closure of the orthogonal sum with additivity. Every hypothesis is therefore semantically active.

4 Lüders update as a partial state interface

For raw matrices the development retains the convenient totalized formula

$$\mathcal{L}_P(\rho) = \mu_\rho(P)^{-1} P\rho P,$$

where Lean's field inverse maps zero to zero. This definition is useful for algebraic equalities, including the degenerate case, but it does not itself promise a state. The state-facing definition is instead

`luedersStateUpdate` : `StateMatrix n` $\to$ `ProjectionEvent n` $\to$ $(\mu_\rho(P) \neq 0)$ $\to$ `StateMatrix n`.

It assumes `NeZero n`; unit trace is impossible for the unique 0×0 matrix, so the exclusion belongs at the state API boundary.

Proposition 1 (Checked Lüders interface)**.** *For a state ρ, event P, and nonzero outcome weight:*

(i) the raw formula is positive semidefinite and has trace one (`luedersUpdate_isState`);

(ii) the updated state assigns weight one to P (`bornWeight_luedersUpdate_self`);

(iii) update is an idempotent retraction (`luedersUpdate_idem`);

(iv) updates by commuting events compose and commute (`luedersUpdate_luedersUpdate_of_commute` and `luedersUpdate_comm`); and

(v) the fixed-state equivalence

$$\mathcal{L}_P(\rho) = \rho \quad \Longleftrightarrow \quad \mu_\rho(P) = 1$$

holds without a separate nonzero-weight hypothesis (`luedersUpdate_eq_self_iff`).

The last statement removes a logically redundant guard. If a state were fixed by a zero-weight totalized update, it would equal the zero matrix, contradicting unit trace. Thus totalization remains visible at the raw layer without leaking an unnecessary precondition into the semantic fixed-point theorem.

5 Bundled arbitrary-partition pinching

A `ProjectivePartition n k` consists of projections $(P_i)_{i<k}$, pairwise orthogonality, and $\sum_i P_i = \mathbf{1}$. It induces

$$\mathcal{E}_\pi(X) = \sum_i P_i X P_i, \qquad N_\pi = \{X : XP_i = P_i X \text{ for every } i\}.$$

The code bundles N_π as `ProjectivePartition.commutant`, a `StarSubalgebra` defined using Mathlib's centralizer, and bundles $\mathcal{E}_\pi$ as `partitionPinchingLinearMap`. The membership theorem `mem_commutant_iff` exposes the expected pointwise equation.

This terminology matters. The range is generally a noncommutative block-diagonal commutant, not a "classical algebra". We make no checked claim that the algebra generated by the partition is the center of this commutant, nor that rank-one partitions yield a formal classical representation.

Theorem 1 (Bundled retraction and exact range)**.** *For every projective partition π, the map $\mathcal{E}_\pi$ is complex linear, unital, positive, trace preserving, and idempotent. It lands in N_π and fixes N_π pointwise. Consequently*

$$\mathcal{E}_\pi(X) = X \Longleftrightarrow X \in N_\pi, \qquad \operatorname{range}(\mathcal{E}_\pi) = N_\pi$$

where the second equality is the equality of the linear-map range and the underlying submodule of the bundled commutant.

The corresponding declarations are `partitionPinching_unital`, `partitionPinching_posSemidef`, `trace_partitionPinching`, `partitionPinching_idem`, `partitionPinching_eq_self_iff_mem_commutant`, and `range_partitionPinchingLinearMap`. State preservation is exposed by `partitionPinching_isState`.

Theorem 2 (Bimodule and Hilbert–Schmidt structure)**.** *If $A, B \in N_\pi$, then*

$$\mathcal{E}_\pi(AXB) = A\mathcal{E}_\pi(X)B.$$

Furthermore, pinching is selfadjoint for the trace pairing, satisfies the Pythagorean identity

$$\operatorname{Tr}(X^*X) = \operatorname{Tr}(\mathcal{E}_\pi(X)^*\mathcal{E}_\pi(X)) + \operatorname{Tr}((X - \mathcal{E}_\pi(X))^*(X - \mathcal{E}_\pi(X))),$$

and hence contracts the squared Hilbert–Schmidt quantity. A commutant-valued complex-linear map with the same trace pairings against every commutant element equals `partitionPinchingLinearMap`.

These results are checked by `partitionPinching_bimodule`, `trace_conjTranspose_partitionPinching_mul`, `trace_partitionPinching_pythagoras`, `trace_partitionPinching_contraction`, and `partitionPinchingLinearMap_unique`. They justify describing the map as a positive, unital, trace-preserving linear projection with a bimodule law. We do not promote it to a bundled operator-algebra conditional expectation, because this artifact does not prove complete positivity or instantiate a library channel interface.

5.1 Lüders naturality

If $P \in N_\pi$, compression commutes with pinching. After normalization, trace preservation against commutant elements gives

$$\mathcal{E}_\pi(\mathcal{L}_P(\rho)) = \mathcal{L}_P(\mathcal{E}_\pi(\rho)).$$

The raw formula is `partitionPinching_luedersUpdate`. More importantly, the theorem `partitionPinching_luedersStateUpdate` states the same naturality for `StateMatrix n` and `ProjectionEvent n`: the input and output are states, and the nonzero-weight proof is transported by `bornWeight_partitionPinching`. This is an end-to-end client of the new bundled and typed interfaces.

6 The bundled state expectation

For a matrix ρ, `stateExpectationLinearMap rho` bundles $M \mapsto \mathrm{Tr}(\rho M)$ as a complex-linear functional. If ρ is a state it maps identity to one. If ρ and M are positive semidefinite, then the expectation is nonnegative. The proof of `expectation_nonneg` uses a finite spectral decomposition of M, cycles the trace, and reduces the result to a sum of products of nonnegative diagonal entries. The former duplicate pointwise additivity and homogeneity lemmas are now inherited from the bundled `LinearMap` interface.

7 CHSH interoperability as a client

For selfadjoint involutions a_0, a_1, b_0, b_1 with cross commutation, set

$$S = a_0b_0 + a_0b_1 + a_1b_0 - a_1b_1.$$

The development first proves, in a bare ring,

$$S^2 = 4\mathbf{1} - (a_0a_1 - a_1a_0)(b_0b_1 - b_1b_0)$$

(`chsh_mul_self`). In a nontrivial unital C*-ring this yields the operator-norm bound $\|S\| \le 2\sqrt{2}$ (`tsirelson_bound`). The precise Mathlib structure is a `NormedRing`, `StarRing`, `CStarRing`, and `Nontrivial`; the manuscript does not call this a nonunital C*-algebra.

The theorem `tsirelson_bound_of_isCHSHTuple` consumes Mathlib's `IsCHSHTuple`. The matrix theorem activates Mathlib's scoped operator norm. Finally, `matrix_tsirelson_bound_of_events` accepts four projection events plus the four cross-commutation equations, constructs the dichotomic observables $\mathbf{1} - 2P$, and returns the norm bound. This makes CHSH a genuine downstream client of the event interface rather than an adjacent theorem inventory.

The result complements rather than supersedes Mathlib's `tsirelson_inequality`, which is an order inequality in a star-ordered real algebra [4]. No state expectation is taken here, and no equality witness is formalized.

8 Artifact evaluation and proof audit

The source is a Lake project pinned to `leanprover/lean4:v4.29.1`; its manifest pins Mathlib commit `5e932f97dd25535344f80f9dd8da3aab83df0fe6`. The library is built with

```
lake build EventAlgebra.
```

The canonical Lean modules are publicly available in the Observer Patch Holography repository [5] at commit `da9ff3a551e7c635370ec885d972c9527de173b0`. The manuscript source accompanies the submission, together with a standalone archive containing a byte-identical copy of those modules. Every public result discussed in this paper is named above. The modules end with `#print axioms` commands for the principal declarations. In the reported build, those commands list only standard Lean/Mathlib logical axioms such as propositional extensionality, quotient soundness, and classical choice; no declaration reports `sorryAx`. A source scan also rejects `sorry` placeholders in the submitted modules.

The audit led to semantic rather than cosmetic changes: an unused-hypothesis theorem was replaced; a sequential-update theorem lost an unused event hypothesis; state update became typed and dimension-aware; the fixed-point equivalence lost a redundant guard; the commutant and pinching became bundled; and the event-to-CHSH adapter became an explicit client theorem. These changes are reflected in the submitted source, not merely in prose.

The release archive includes the toolchain, Lake manifest, source modules, build command, checksums, and captured axiom output. A public tagged release and persistent archive identifier are desirable publication steps but are not claimed until such identifiers exist.

9 Feature-level comparison

Table 2 states the positioning at the level of compiled features. This avoids the misleading claim that the collection of elementary results is itself unprecedented.

Physlib is strictly stronger on channel structure: its spectral pinching is a `CPTPMap` [6]. The present artifact is more general only along a different axis: the partition is supplied directly and need not be extracted from the spectrum of a chosen state. Proving a correspondence theorem for spectral partitions would make these axes interoperate; until then, they remain separate APIs.

The broad Lean-Quantum and Lean-QIT projects cover substantially more quantum information than the present finite measurement kernel [3, 8]. The purpose here is a small, audited bridge among projections, commutants, pinching, selective update, and a Mathlib CHSH client, not an alternative full-stack library.

Feature	Closest checked infrastructure	This artifact
Projective measurement	Isabelle/HOL projective-measurement developments; state/measurement interfaces in current Lean libraries	finite matrix event predicate, typed event/state subtypes, closure and trace-pairing lemmas; treated as prerequisite rather than priority claim
Pinching	Physlib spectral pinching is a state-selected bundled CPTP map with fixed-point and geometric results	arbitrary supplied projective partition; bundled linear map and commutant; no CPTP or Physlib correspondence theorem
Retraction structure	operator-algebra conditional expectations and channel interfaces	exact range/fixed points, positive/unital/trace-preserving, bimodule, Hilbert–Schmidt geometry, map-level trace-duality uniqueness
Lüders update	Isabelle projective measurement and general channel infrastructure	raw formula plus typed partial state API, unguarded state fixed-point equivalence, typed naturality with arbitrary-partition pinching
CHSH/Tsirelson	Mathlib order theorem; Isabelle upper bound; Lean CHSH rigidity	complementary norm theorem and event-level client; no state-level bound or tightness claim
Reproducibility	public CI and archived releases are common best practice	pinned Lean/Mathlib, clean artifact build instructions, checksums and static axiom audit; no DOI claim before deposit

Table 2: Direct comparison with the closest formal infrastructure.

10 Limitations and conclusion

We verified a finite-matrix interface connecting projection events, trace-based probabilities, a typed Lüders state update, arbitrary projective partitions, a bundled block pinching, and an event-level CHSH norm client. The central result is interoperability: the pinching map has exactly the bundled commutant as its range, obeys its bimodule law, and is natural with typed Lüders update for commutant events. This moves the development beyond a list of elementary matrix lemmas while keeping every interpretive claim attached to a named declaration.

Several limitations are deliberate. Complete positivity is not established, so the pinching is not yet exported as a quantum channel. The code does not identify the construction with Physlib's spectral pinching. It has no formal rank-one classical specialization. The CHSH client stops at an operator-norm bound, without a state-expectation corollary or sharpness witness. Finally, the development is finite-dimensional; it does not address normal maps or infinite-dimensional von Neumann algebras.

The most valuable next step is therefore interoperability, not another parallel theorem inventory: prove complete positivity, construct a bundled CPTP map, connect spectral partitions to Physlib, and add a state-level client that consumes the resulting channel and expectation interfaces. The current artifact supplies the arbitrary-partition, exact-range, bimodule, and typed naturality kernel on which that extension can be based.

Declarations

Funding. No funding was received for this work.

Competing interests. The author declares no competing interests.

Author contributions. Bernhard Mueller is the sole author, designed the study, reviewed the formal development, and takes responsibility for the manuscript and artifact.

Data and code availability. No empirical data were generated. The Lean source, pinned dependency files, build instructions, axiom report, and checksums accompany the submission. Public release identifiers should be inserted only after the corresponding deposit exists.

Use of generative AI. Generative-AI tools assisted with drafting, proof suggestions, and code review. The author reviewed and tested the resulting manuscript and source and takes full responsibility for them. AI tools are not authors; acceptance of Lean proofs was checked by the pinned Lean kernel.

References

[1] Mnacho Echenim. Quantum projective measurements and the CHSH inequality. *Archive of Formal Proofs*, 2021. Formal proof development, ISSN 2150-914x.

[2] Mnacho Echenim and Mehdi Mhalla. A formalization of the CHSH inequality and Tsirelson's upper-bound in Isabelle/HOL. *Journal of Automated Reasoning*, 68:Article 2, 2024. DOI: 10.1007/s10817-023-09689-9.

[3] Kazumi Kasaura, Kei Tsukamoto, Kento Mori, Risa Mizuno, Takahiro Namatame, Yuta Oriike, Masaya Taniguchi, Sho Sonoda, and Hayata Yamasaki. Lean-Quantum: Toward AI-assisted formalization of quantum information, 2026.

[4] Kim Morrison and Mathlib contributors. Mathlib.algebra.star.chsh. Mathlib documentation, 2026. URL https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/Star/CHSH.html. Accessed 20 July 2026.

[5] Observer Patch Holography contributors. Observer Patch Holography: Source repository. https://github.com/FloatingPragma/observer-patch-holography, 2026. Commit da9ff3a551e7c635370ec885d972c9527de173b0; accessed 20 July 2026.

[6] Physlib contributors. Quantuminfo.finite.pinching: Pinching channels. Lean library documentation, 2026. URL https://ohaithe.re/Lean-QuantumInfo/QuantumInfo/Finite/Pinching.html. Accessed 20 July 2026.

[7] Tianrun Zhao and Nengkun Yu. Formalizing CHSH rigidity in Lean 4, 2026.

[8] Chengkai Zhu, Ziao Tang, Guocheng Zhen, Yimeng Cao, Yusheng Zhao, Ranyiliu Chen, Xuanqiang Zhao, Lei Zhang, and Xin Wang. Lean-QIT: Towards a formal infrastructure for quantum information theory, 2026.